

46. 全排列

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 数组, 回溯
- **Companies:** (Not Available)

给定一个不含重复数字的数组 `nums`，返回其 所有可能的全排列。你可以 按任意顺序返回答案。

示例 1:

输入: `nums = [1,2,3]`

输出: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

示例 2:

输入: `nums = [0,1]`

输出: `[[0,1],[1,0]]`

示例 3:

输入: `nums = [1]`

输出: `[[1]]`

提示:

- `1 <= nums.length <= 6`
- `-10 <= nums[i] <= 10`
- `nums` 中的所有整数 互不相同

Discussion | Solution

解决方法

1. 问题理解

目标: 生成给定数组（不含重复数字）的所有可能排列。输入: 一个整数数组 `nums`。
输出: 一个包含 `nums` 所有全排列的列表。

全排列: 将数组中的元素进行重新排序, 得到所有可能的顺序。对于长度为 `n` 的数组, 共有 `n!` 种全排列。

2. 思路分析

这是典型的回溯算法应用场景。回溯算法可以看作是在一个决策树上进行深度优先搜索的过程。

决策树模型：- 树的每一层代表排列中的一个位置。- 每一层的节点代表在该位置可以选择的数字。- 从根节点到叶子节点的路径构成一个完整的排列。

回溯过程：

1. 路径 (**Path**)：记录当前已经选择的数字构成的部分排列。
2. 选择列表 (**Choices**)：当前位置可以从哪些数字中选择。关键在于，已经选择过的数字不能再次选择。
3. 结束条件 (**End Condition**)：当路径的长度等于数组 `nums` 的长度时，说明找到了一个完整的排列，将其加入结果列表。

具体步骤：

1. 定义回溯函数 `backtrack(path, used)`：
 - `path`：当前构建的部分排列（一个列表）。
 - `used`：一个布尔数组或集合，标记 `nums` 中的哪些元素已经被使用过。
2. 递归终止条件：
 - 如果 `len(path) == len(nums)`，说明 `path` 已经是一个完整的排列。
 - 将 `path` 的一个副本（因为 `path` 会在后续回溯中被修改）加入结果列表 `result`。
 - 返回。
3. 遍历选择列表：
 - 遍历 `nums` 数组中的每个元素 `nums[i]`（索引从 0 到 `len(nums) - 1`）。
 - 剪枝/判断选择是否有效：检查 `nums[i]` 是否已经被使用过（即 `used[i]` 是否为 `True`）。
 - 如果 `nums[i]` 没有被使用过：
 - 做选择：
 - * 将 `nums[i]` 添加到当前路径 `path`。
 - * 将 `used[i]` 标记为 `True`。
 - 递归进入下一层决策：调用 `backtrack(path, used)`。
 - 撤销选择（回溯）：
 - * 将 `nums[i]` 从 `path` 末尾移除 (`path.pop()`)。
 - * 将 `used[i]` 标记为 `False`（恢复状态，以便其他分支可以使用 `nums[i]`）。
4. 主函数调用：
 - 初始化结果列表 `result = []`。
 - 初始化 `used` 数组（全 `False`）。
 - 调用 `backtrack([], used)` 开始回溯。
 - 返回 `result`。

优化/替代方案：

- 不使用 `used` 数组，通过交换元素：可以在递归过程中直接修改 `nums` 数组。在第 `k` 层递归时，将 `nums[k]` 与 `nums[k]` 到 `nums[n-1]` 中的每一个元素

交换，然后递归到下一层 $k+1$ 。递归返回后，再交换回来以恢复状态。这种方法可以省去 `used` 数组的空间，但会修改原始输入（如果需要保留原始输入则需先复制）。

```
def backtrack_swap(index):
    if index == len(nums):
        result.append(nums[:]) # 添加副本
        return
    for i in range(index, len(nums)):
        nums[index], nums[i] = nums[i], nums[index] # 做选择（交换）
        backtrack_swap(index + 1) # 递归
        nums[index], nums[i] = nums[i], nums[index] # 撤销选择（换回来）
```

3. 代码实现 (Python)

```
from typing import List

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        """
        使用回溯算法生成给定数组的全排列。
        Args:
            nums: 不含重复数字的整数数组。
        Returns:
            包含 nums 所有全排列的列表。
        """
        n = len(nums)
        result = [] # 用于存储所有排列结果
        path = [] # 用于存储当前构建的排列路径
        used = [False] * n # 标记 nums 中的元素是否已被使用

    def backtrack():
        """
        回溯函数核心逻辑。
        """
        # 递归终止条件：当路径长度等于数组长度时，找到一个完整排列
        if len(path) == n:
            # 将当前路径的副本加入结果列表
            # 必须是副本 (path[:])，因为 path 在后续回溯中会被修改
            result.append(path[:])
            return

        # 遍历 nums 中的所有元素，尝试将其加入当前排列
        for i in range(n):
            # 剪枝：如果 nums[i] 已经被使用过，则跳过
            if used[i]:
```

```

        continue

    # --- 做选择 ---
    path.append(nums[i]) # 将 nums[i] 加入当前路径
    used[i] = True      # 标记 nums[i] 为已使用

    # --- 递归进入下一层决策 ---
    backtrack()

    # --- 撤销选择 (回溯) ---
    path.pop()          # 将 nums[i] 从路径末尾移除
    used[i] = False     # 标记 nums[i] 为未使用, 恢复状态

# 从空路径开始调用回溯函数
backtrack()
return result

# --- 使用交换元素的优化方法 (可选) ---
def permute_swap(self, nums: List[int]) -> List[List[int]]:
    """
    使用回溯算法和元素交换生成全排列 (原地修改 nums)。
    """
    n = len(nums)
    result = []

    def backtrack_swap(index: int):
        """
        回溯函数, 通过交换元素来构建排列。
        Args:
            index: 当前处理到的排列位置 (从 0 开始)。
        """
        # 递归终止条件: 当处理到最后一个位置之后时, 形成一个完整排列
        if index == n:
            result.append(nums[:]) # 添加当前 nums 状态的副本
            return

        # 遍历从当前位置 index 到末尾的所有元素
        for i in range(index, n):
            # 将第 i 个元素换到当前 index 位置
            nums[index], nums[i] = nums[i], nums[index]
            # 递归处理下一个位置 index + 1
            backtrack_swap(index + 1)
            # 撤销交换, 恢复 nums 状态, 以便进行其他分支的交换
            nums[index], nums[i] = nums[i], nums[index]

    # 从第 0 个位置开始回溯

```

```

        backtrack_swap(0)
    return result

# 示例调用
# sol = Solution()
# print(sol.permute([1, 2, 3]))
# print(sol.permute_swap([1, 2, 3]))

```

4. 复杂度分析

设数组 `nums` 的长度为 N 。

- 时间复杂度： $O(N * N!)$
 - 生成一个排列需要 $O(N)$ 的时间（路径长度为 N ）。
 - 总共有 $N!$ 个不同的排列。
 - 回溯过程中，虽然节点总数远超 $N!$ ，但到达叶子节点（生成一个完整排列）的总次数是 $N!$ 次。每次到达叶子节点需要 $O(N)$ 时间复制路径。中间节点的计算可以分摊到叶子节点上。更严谨的分析是，状态树大约有 $N!$ 个叶子节点，树的总节点数约为 $O(N!)$ 级别（更精确是 $\sum(N! / k!)$ for k from 0 to $N-1$ ），每个节点的操作是 $O(N)$ （遍历选择列表）或 $O(1)$ （交换法），复制结果是 $O(N)$ 。所以总体是 $O(N * N!)$ 。
- 空间复杂度： $O(N)$
 - 递归调用栈的深度最多为 N 。
 - `path` 列表或 `used` 数组需要 $O(N)$ 的空间。
 - 结果列表 `result` 需要存储 $N!$ 个长度为 N 的列表，空间为 $O(N * N!)$ ，但这通常不计入算法本身的额外空间复杂度。如果计入，则空间复杂度为 $O(N * N!)$ 。

5. 如何在面试中讲解

1. 问题定义：
 - “目标是找出给定数组（无重复元素）的所有排列组合。”
2. 核心思路：回溯：
 - “这是一个典型的回溯问题。我们可以想象成构建一个排列的过程，每次决定在当前位置放哪个数字。”
 - “使用回溯算法，通过深度优先搜索来探索所有可能的排列。”
3. 回溯三要素：
 - “路径 (**Path**): 用一个列表记录当前已经选择的数字序列。”
 - “选择列表 (**Choices**): 在当前位置，可以从 `nums` 中选择哪些还未使用过的数字。”
 - “结束条件: 当路径的长度等于 `nums` 的长度时，表示找到了一个完整的排列，将其加入结果集。”
4. 实现细节 (使用 `used` 数组):
 - “定义一个递归函数 `backtrack()`。”
 - “函数内部，首先检查是否满足结束条件（路径长度达标），如果满足，复制当前路径到结果集并返回。”
 - “然后，遍历 `nums` 中的每个数字 `num`。”

- “使用一个 `used` 布尔数组来判断 `num` 是否已经被用在当前路径中。如果已使用，跳过。”
 - “如果未使用：做选择（将 `num` 加入 `path`，标记 `used`），递归调用 `backtrack()`，撤销选择（从 `path` 移除 `num`，取消标记 `used`）。”
 - “主函数初始化结果集、`used` 数组，然后调用 `backtrack()`。”
5. (可选) 实现细节 (交换法):
- “另一种方法是直接在 `nums` 数组上操作。定义 `backtrack(index)` 表示确定第 `index` 个位置的元素。”
 - “在 `backtrack(index)` 中，遍历从 `index` 到末尾的所有元素 `i`。”
 - “将 `nums[i]` 与 `nums[index]` 交换（相当于选择 `nums[i]` 放在第 `index` 位）。”
 - “递归调用 `backtrack(index + 1)`。”
 - “递归返回后，必须将 `nums[i]` 和 `nums[index]` 交换回来，以恢复状态，确保不影响其他分支的搜索。”
 - “结束条件是 `index == len(nums)`。”
 - “这种方法省去了 `used` 数组，但会修改输入数组（如果需要保留原数组需先复制）。”
6. 复杂度分析:
- “时间复杂度是 $O(N * N!)$ ，因为有 $N!$ 个排列，生成每个排列需要 $O(N)$ 时间（主要是复制路径）。”
 - “空间复杂度是 $O(N)$ ，主要是递归栈的深度和辅助数据结构（`path` 或 `used`）。”